

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

Department of Artificial Intelligence

Intelligent Multi-Agent and Expert Systems (AI401)

Semester VII

Experiment 2

Production Rules and the Match Phase

Name Aditya Gaur

Roll Number U23AI095

1. Aim

To represent domain knowledge as IF–THEN production rules, hold facts in a working memory, and compute the conflict set of applicable rules.

2. Theory

A production system consists of three major components: a rule base, working memory, and an inference engine.

The **rule base** contains IF–THEN production rules of the form

IF conditions THEN conclusion.

The **Working Memory (WM)** contains the facts that are currently believed or available to the inference engine.

The **inference engine** performs the recognise–act cycle. It matches the rules against the current working memory, selects a rule from the resulting conflict set, executes its conclusion, and repeats the process.

This experiment specifically focuses on the **match phase**. A rule is applicable when all conditions in its IF part are satisfied by the current working memory and its conclusion is not already present in working memory. The latter condition is called **refractoriness**; it prevents the same rule from repeatedly firing on the same data.

The set of all currently applicable rules is called the **conflict set**. Rule selection from this set belongs to a later stage of the expert-system process.

3. Knowledge Base

A smart-home security and automation domain was used for the practical. The knowledge was represented using IF-THEN production rules.

The production rules used are:

Rule	IF conditions	THEN conclusion
R1	person_detected	camera_alert
R2	person_detected AND home_armed	security_alert
R3	door_open AND home_armed	door_alarm
R4	motion_detected AND night_mode	lights_on
R5	smoke_detected	fire_alarm
R6	water_leak AND NOT maintenance_mode	water_alarm
R7	package_detected AND NOT package_collected	delivery_alert
R8	window_open AND NOT home_occupied	window_alert
R9	security_alert	notify_owner
R10	camera_alert AND NOT privacy_mode	record_video
R11	home_armed AND NOT maintenance_mode	armed_monitoring
R12	battery_low	low_battery_alert

4. Working Memory and Match Phase

Working memory is represented as a set of current facts. The initial working memory used in the main demonstration is

$$\{\text{home_armed}, \text{person_detected}\}$$

For every rule, the matcher checks whether:

1. every positive condition is present in working memory;
2. every negated condition is satisfied, meaning that the corresponding fact is absent from working memory; and
3. the conclusion is not already present in working memory.

The rules satisfying these conditions form the conflict set.

For the initial working memory,

$$WM = \{\text{home_armed}, \text{person_detected}\},$$

the conflict set is

$$\{R1, R2, R11\}.$$

R1 matches because `person_detected` is present. R2 matches because both `person_detected` and `home_armed` are present. R11 matches because `home_armed` is present and `maintenance_mode` is absent.

5. Algorithm

1. Encode each rule as a record containing an identifier, a list of conditions, and a conclusion.
2. Initialise working memory as a set of facts.
3. For each rule, test whether every positive condition belongs to working memory and every negated condition refers to a fact absent from working memory.
4. Check refractoriness by verifying that the rule conclusion is not already present in working memory.
5. If all conditions are satisfied and the conclusion is absent, add the rule to the conflict set.
6. Display the resulting conflict set.
7. Modify working memory and repeat the matching process to observe how the conflict set changes.

6. Procedure

1. Define the domain knowledge using IF–THEN production rules.
2. Store the currently known facts in working memory.
3. Compare the conditions of every production rule with the facts present in working memory.
4. If all conditions of a rule are satisfied, classify the rule as a matching rule.
5. Collect all matching rules into the conflict set.
6. Run the program using the initial working-memory facts and observe the conflict set.
7. Add or modify facts in working memory and observe how the conflict set grows or shrinks.
8. Test negated conditions using rules containing `NOT fact`.
9. Measure matching time as the number of automatically generated rules is increased to 1,000 and relate the result to the RETE algorithm.

7. Procedure and Result

The production-rule program was executed successfully. The output shows the complete knowledge base, the initial working memory, the initial conflict set, and detailed explanations for why individual rules match or do not match.

```
PS C:\Users\adity\Desktop\Sem7> & "c:\Program Files\Python313\python.exe" c:/Users/adity/Desktop/Sem7/IMAESLab/lab02/U23AI095_IMAES_LAB02.py
AI401 – EXPERIMENT 2
Production Rules and the Match Phase

=====
KNOWLEDGE BASE – IF-THEN PRODUCTION RULES
=====
R1: IF person_detected THEN camera_alert
R2: IF person_detected AND home_armed THEN security_alert
R3: IF door_open AND home_armed THEN door_alarm
R4: IF motion_detected AND night_mode THEN lights_on
R5: IF smoke_detected THEN fire_alarm
R6: IF water_leak AND NOT maintenance_mode THEN water_alarm
R7: IF package_detected AND NOT package_collected THEN delivery_alert
R8: IF window_open AND NOT home_occupied THEN window_alert
R9: IF security_alert THEN notify_owner
R10: IF camera_alert AND NOT privacy_mode THEN record_video
R11: IF home_armed AND NOT maintenance_mode THEN armed_monitoring
R12: IF battery_low THEN low_battery_alert

Initial working memory: {'home_armed', 'person_detected'}
Conflict set: ['R1', 'R2', 'R11']

Detailed match explanations:
R1: MATCHED – all conditions satisfied.
R2: MATCHED – all conditions satisfied.
R3: NOT MATCHED – missing positive condition(s): door_open
R4: NOT MATCHED – missing positive condition(s): motion_detected, night_mode
R5: NOT MATCHED – missing positive condition(s): smoke_detected
R6: NOT MATCHED – missing positive condition(s): water_leak
R7: NOT MATCHED – missing positive condition(s): package_detected
R8: NOT MATCHED – missing positive condition(s): window_open
R9: NOT MATCHED – missing positive condition(s): security_alert
R10: NOT MATCHED – missing positive condition(s): camera_alert
R11: MATCHED – all conditions satisfied.
R12: NOT MATCHED – missing positive condition(s): battery_low
```

Figure 1: Production-rule knowledge base, initial working memory, conflict set, and detailed matching results.

The initial result is:

Item	Result
Working Memory	{home_armed, person_detected}
Conflict Set	[R1, R2, R11]

The detailed explanations also show why rules fail to match. For example, R3 is not matched because `door_open` is missing, while R11 matches because its positive condition `home_armed` is present and its negative condition `NOT maintenance_mode` is satisfied.

8. Exercise 1 – Working Memory Updated One Fact at a Time

The working memory was modified by adding one fact at a time. After each modification, the complete conflict set was recomputed.

```

=====
EXERCISE 1 – WORKING MEMORY UPDATED ONE FACT AT A TIME
=====

```

Step	Added fact	Conflict set
1	home_armed	['R11']
2	person_detected	['R1', 'R2', 'R11']
3	door_open	['R1', 'R2', 'R3', 'R11']
4	motion_detected	['R1', 'R2', 'R3', 'R11']
5	night_mode	['R1', 'R2', 'R3', 'R4', 'R11']
6	maintenance_mode	['R1', 'R2', 'R3', 'R4']
7	water_leak	['R1', 'R2', 'R3', 'R4']
8	package_detected	['R1', 'R2', 'R3', 'R4', 'R7']
9	package_collected	['R1', 'R2', 'R3', 'R4']
10	privacy_mode	['R1', 'R2', 'R3', 'R4']
11	smoke_detected	['R1', 'R2', 'R3', 'R4', 'R5']
12	battery_low	['R1', 'R2', 'R3', 'R4', 'R5', 'R12']

Observation:
The conflict set changes after each WM update. Adding a required positive fact can activate a rule; adding a fact that violates a NOT-condition can deactivate a rule. Adding a conclusion already required by a matching rule demonstrates refractoriness.

Figure 2: Exercise 1: conflict-set changes after adding facts one at a time.

The observed sequence is:

Step	Fact Added	Conflict Set
1	home_armed	R11
2	person_detected	R1, R2, R11
3	door_open	R1, R2, R3, R11
4	motion_detected	R1, R2, R3, R11
5	night_mode	R1, R2, R3, R4, R11
6	maintenance_mode	R1, R2, R3, R4
7	water_leak	R1, R2, R3, R4
8	package_detected	R1, R2, R3, R4, R7
9	package_collected	R1, R2, R3, R4
10	privacy_mode	R1, R2, R3, R4
11	smoke_detected	R1, R2, R3, R4, R5
12	battery_low	R1, R2, R3, R4, R5, R12

This experiment demonstrates that the conflict set is dynamic. A new positive fact may satisfy a previously missing condition and activate a rule. Conversely, adding a fact that violates a NOT condition can deactivate a rule.

For example, before step 6, R11 is applicable because

$$\text{home_armed} \in WM$$

and

$$\text{maintenance_mode} \notin WM.$$

After adding `maintenance_mode`, the condition

$$\text{NOT maintenance_mode}$$

becomes false, so R11 leaves the conflict set.

9. Exercise 2 – Negated Conditions

The matcher was extended to support conditions of the form

$$\text{NOT fact.}$$

A negated condition is satisfied only when the corresponding fact is absent from working memory.

For example,

R6: IF `water_leak` AND NOT `maintenance_mode` THEN `water_alarm`.

Initially, if `water_leak` is present and `maintenance_mode` is absent, R6 matches.

After adding `maintenance_mode`, the negative condition becomes false and the rule is removed from the conflict set.

The same mechanism is demonstrated using the package-delivery and privacy-mode rules.

```

=====
EXERCISE 2 – NEGATED CONDITIONS
=====
Initial WM: ['camera_alert', 'package_detected', 'water_leak']
Initial conflict set: ['N1', 'N2', 'N3']

After adding 'maintenance_mode':
Conflict set: ['N2', 'N3']
N1 explanation: N1: NOT MATCHED – negated condition(s) violated: maintenance_mode

After adding 'package_collected':
Conflict set: ['N3']
N2 explanation: N2: NOT MATCHED – negated condition(s) violated: package_collected

After adding 'privacy_mode':
Conflict set: []
N3 explanation: N3: NOT MATCHED – negated condition(s) violated: privacy_mode

=====
REFRACTORINESS DEMONSTRATION
=====
WM before conclusion is added: ['person_detected']
Conflict set: ['RF1']

WM after adding the conclusion 'camera_alert': ['camera_alert', 'person_detected']
Conflict set: []
RF1: NOT MATCHED – conclusion already in WM (refractory).

```

Figure 3: Exercise 2: negated conditions and refractoriness demonstration.

The observed changes are:

Working-memory change	Conflict Set
Initial WM	[N1, N2, N3]
After adding maintenance_mode	[N2, N3]
After adding package_collected	[N3]
After adding privacy_mode	[]

Thus, adding a fact that violates a negative condition can remove a previously matching rule.

9.1 Refractoriness

Refractoriness prevents a rule from remaining applicable once its conclusion is already present in working memory.

For example,

R1: IF person_detected THEN camera_alert.

Initially,

$$WM = \{\text{person_detected}\}$$

and R1 belongs to the conflict set.

After adding its conclusion,

$$\text{camera_alert},$$

the working memory becomes

$$WM = \{\text{person_detected}, \text{camera_alert}\}.$$

The condition of R1 is still satisfied, but its conclusion is already present. Therefore, R1 becomes refractory and is removed from the conflict set.

The screenshot verifies:

$$\text{Conflict set} = []$$

with the explanation:

$$\text{conclusion already in WM (refractory)}.$$

10. Exercise 3 – Match-Time Scaling

The matcher was benchmarked using Python's `time.perf_counter()` while automatically scaling the number of rules to:

$$100, 250, 500, 1000.$$

The generated rules contain multiple conditions, and the benchmark repeatedly performs a complete match pass over the rule base.

```

=====
EXERCISE 3 – MATCH-TIME SCALING
=====
Rules          Average match time (ms)
-----
100             0.048663
250             0.140316
500             0.263995
1000            0.508940

Interpretation:
A naive production-system matcher scans the rule base and tests
conditions for every cycle. Therefore, as the number of rules grows,
the amount of work grows approximately with the number of rules
(and with the number of conditions per rule).

Why RETE exists:
RETE avoids recomputing all rule-condition matches from scratch after
every WM change. It shares common condition tests and maintains
partial matches in an incremental network, substantially reducing
repeated work in large production systems.

=====
CORRECTNESS TESTS
=====
All correctness tests passed.

=====
FINAL RESULT: EXPERIMENT 2 COMPLETED SUCCESSFULLY
=====
PS C:\Users\adity\Desktop\Sem7>

```

Figure 4: Exercise 3: match-time scaling, RETE interpretation, and final correctness result.

The recorded average match times from the practical are:

Number of Rules	Average Match Time (ms)
100	0.048663
250	0.140316
500	0.263995
1000	0.508940

The results show that the matching time increases as the rule base grows.

10.1 Interpretation of Scaling

The implemented matcher uses a straightforward strategy:

For every rule $\rightarrow$ check its conditions against WM.

Therefore, as the number of rules increases, more rule-condition tests must be performed.

If there are R rules and an average of C conditions per rule, the basic matching work is approximately

$$O(RC).$$

For a fixed average number of conditions, this is approximately

$$O(R)$$

with respect to the number of rules.

The benchmark therefore demonstrates why a naive production-system matcher becomes increasingly expensive as the knowledge base grows.

10.2 Why the RETE Algorithm Exists

The **RETE** algorithm is a pattern-matching approach for production-rule systems designed to reduce repeated matching work.

In a naive matcher, a change in working memory may cause the system to recheck every rule and every condition from the beginning.

RETE instead uses an incremental matching network that:

- shares common condition tests between rules;
- stores partial matches;
- updates only the affected parts of the matching network when working memory changes; and
- avoids recomputing the same rule-condition matches from scratch.

Therefore, the relationship demonstrated by this experiment is:

More Rules $\rightarrow$ More Naive Matching Work $\rightarrow$ Higher Match Cost

which motivates

RETE $\rightarrow$ Incremental and Shared Matching.

11. Observations

1. The conflict set changes whenever the contents of working memory change because the applicability of rules depends directly on the current facts.
2. Adding a required positive fact can activate a rule. For example, adding `person_detected` makes R1 and R2 applicable when their other conditions are also satisfied.
3. Adding a fact that violates a NOT condition can deactivate a previously matching rule. For example, adding `maintenance_mode` makes R11 inapplicable.
4. A rule with a missing positive condition does not belong to the conflict set.
5. Refractoriness prevents repeated applicability of a rule when its conclusion is already present in working memory.
6. The negated-condition experiment shows that the absence of a fact can be an important part of a rule's applicability.
7. The benchmark demonstrates increasing match time with increasing numbers of rules.
8. The scaling behavior motivates incremental matching methods such as RETE, which reduce repeated rule-condition evaluation.

12. Conclusion

The experiment successfully represented domain knowledge using IF-THEN production rules, maintained facts in working memory, and computed the conflict set during the match phase.

The practical demonstrated how the conflict set changes when facts are added one at a time, implemented negated conditions of the form `NOT fact`, and demonstrated refractoriness by preventing a rule from remaining applicable after its conclusion had already been added to working memory.

The benchmark using `time.perf_counter()` showed that naive matching time increases as the number of rules grows from 100 to 1,000. This provides the motivation for RETE-style incremental and shared matching techniques.

The final implementation completed successfully, and all correctness tests passed.